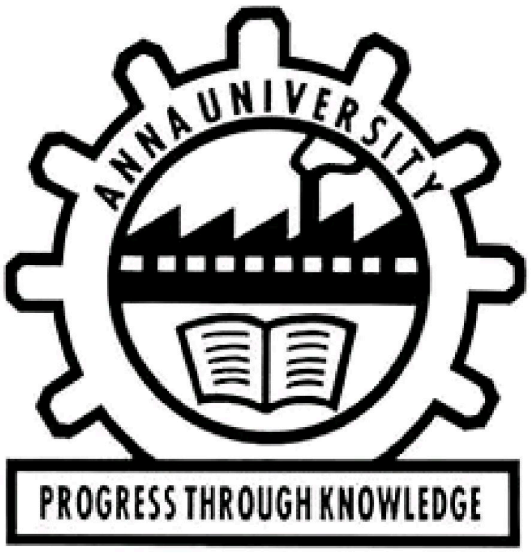
ONLINE ELECTION SYSTEM

A SUMMER INTERNSHIP REPORT

Submitted by

JEYA SANORA FRESHIA J - 810023205703



DEPARTMENT OF INFORMATION TECHNOLOGY

ANNA UNIVERSITY ,

TIRUCHIRAPALLI – 620 024.

UNIVERSITY COLLEGE OF ENGINEERING (BIT CAMPUS),

ANNA UNIVERSITY , TIRUCHIRAPALLI – 620 024

DEPARTMENT OF INFORMATION TECHNOLOGY

BONAFIDE CERTIFICATE

This is to certify this Summer Internship project report **"Online Election System"** is the Bonafide work of **JEYA SANORA FRESHIA J (810023205703)** has satisfactory completed the course of **IT3711 – SUMMER INTERNSHIP** for **SEVENTH** semester, **B.TECH INFORMATION TECHNOLOGY** during the academic year **2025-2026.**

Faculty Incharge **Head of the Department**

Submitted for the practical examination held on _____.

ABSTRACT

The Online Election System is a web-based application developed to simplify and modernize the election process through a digital platform. The system provides a centralized environment where eligible voters can register, log in, view available candidates, and cast their votes electronically. The system also provides administrative functionality for managing voters, candidates, election information, and voting records.

The main objective of the Online Election System is to reduce the limitations associated with traditional election procedures. Manual election processes generally require considerable paperwork, physical presence, manual verification, and extensive effort for maintaining voting records. The proposed system digitizes these activities and provides a convenient platform for conducting elections in an organized manner.

The frontend of the system is developed using web technologies such as HTML, CSS, and JavaScript. The backend is implemented using Node.js and Express.js to handle application logic, user requests, authentication, and communication with the database. MongoDB is used for storing and managing application data such as voter information, candidate information, and voting records.

The system contains important modules including voter registration and login, candidate management, voter management, voting, election management, and administrative management. The system is designed to provide a simple and user-friendly interface while maintaining the accuracy of election records.

Overall, the Online Election System demonstrates the practical application of web development, backend programming, database management, authentication, and frontend-backend integration. The project provides a digital solution for managing election activities efficiently and can be further enhanced with additional security and authentication mechanisms in future versions.

The system is designed to improve the overall efficiency of election management by reducing manual work and providing easy access to election-related information. It helps administrators organize voter and candidate details while allowing voters to participate through a simple online interface. The project also provides practical experience in integrating frontend interfaces with backend services and database operations. The developed system can serve as a basic model for implementing digital election management in educational institutions and organizations, with further improvements possible in security, authentication, scalability, and result management.

This case study also highlights the importance of early detection, regular monitoring, patient education, and proper management in reducing the risk of diabetes-related complications. Effective control of blood glucose, blood pressure, and lipid levels, along with appropriate dietary habits, physical activity, medication adherence, and regular screening of the eyes, kidneys, nerves, and feet, can help prevent or delay disease progression. A coordinated approach involving the patient and healthcare team is essential for improving overall health and maintaining a better quality of life.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
1	INTRODUCTION 1.1 OBJECTIVE OF THE INTERNSHIP	1 - 3
2	COMPANY PROFILE	3-6
3	DAILY WORK REPORT	6 -8
4	BLOCK DIAGRAM	8
5	TRAVEL AGENCY MANAGEMENT SYSTEM 5.1 EXISTING SYSTEM 5.2 PROPOSED SYSTEM 5.3 CAPABILIITIES	8-12
6	PROPOSED METHOD 6.1 USER REGISTRATION 6.2 USER AUTHENTICATION 6.3 CANDIDATE MANAGEMENT 6.4 VOTING MANAGEMENT 6.5 VOTING VALIDATION 6.6 ADMIN MANAGEMENT 6.7 DATABASE MANAGEMENT 6.8 RESULT MANAGEMENT	12 - 14
7	ALGORITHM 7.1 USER REGISTRATION 7.2 USER LOGIN 7.3 CANDIDATE VIEWING 7.4 VOTE CASTING 7.5 VOTE VALIDATION 7.6 VOTE STORAGE 7.7 ADMIN OPERATION	14 - 18
8	CODE IMPLEMENTATION	14 - 28
9	SAMPLE OUTPUT	29
10	ANALYSIS	30
11	CONCLUSION	30
12	FUTURE WORK	30-31
13	REFERENCES	31

1.INTRODUCTION

The Online Election System is a web-based application developed to automate and simplify the process of conducting elections. Elections are an important method of selecting representatives in educational institutions, organizations, and other groups. Traditional election systems generally depend on paper-based voting, manual verification, physical presence of voters, and manual counting of votes.Traditional election procedures can create several difficulties such as long waiting times, manual record maintenance, paperwork, difficulty in managing voter information, and errors during vote counting and record keeping. To overcome these challenges, a centralized web-based Online Election System has been developed.

The primary purpose of this project is to provide a secure and user-friendly platform where voters can register, log in, view candidate information, and cast their electronically. The system also provides an administrator interface for managing voters, candidates, election information, and voting records.

The application is developed using web technologies. The frontend is designed using HTML, CSS, and JavaScript, which provide an interactive and user-friendly interface. The backend is developed using Node.js and Express.js, which handle application logic, user requests, authentication, voting operations, and communication with the database. MongoDB is used as the database to store voter details, candidate information, election information, and voting records.

The system follows a client-server architecture consisting of the Presentation Layer, Application/Backend Layer, and Database Layer. The frontend communicates with the Node.js and Express.js backend through HTTP requests, while the backend communicates with MongoDB for storing and retrieving data.

The Online Election System consists of several important modules, including User Registration and Login, Voter Management, Candidate Management, Election Management, Voting Management, and Admin Management. Each module performs specific functions and works together to provide a complete online election solution.

Overall, the Online Election System is a practical implementation of web application development that demonstrates frontend development, backend development, database connectivity, authentication, API communication, validation, and software testing. The system provides an efficient digital platform for managing election activities.

The Online Election System also focuses on improving the management of election-related information through a centralized digital platform. In a traditional election process, information about voters, candidates, and voting records may be maintained manually, which can make searching, updating, and maintaining records more difficult. The proposed system organizes these details within a structured application and database, allowing authorized users to access the required information more efficiently. The voter can interact with the system through the web interface, while the backend processes the requests and communicates with the database. This integration of frontend, backend, and database components provides a systematic approach to handling election activities. The system also includes validation mechanisms to ensure that the information entered by users follows the required format and that voting operations are performed according to the implemented rules.

Another important aspect of the Online Election System is its ability to provide separate functionalities for voters and administrators. Voters can use the system to register, log in, view available candidates, and participate in the election through the voting interface. Administrators can manage voter information, candidate details, election information, and other authorized operations through the administrative section. The

development of this project provided practical exposure to various stages of web application development, including requirement analysis, interface design, backend implementation, database integration, testing, and debugging. Through this project, the concepts learned during academic studies were applied to a practical application, providing a better understanding of how different software components work together to develop a complete web-based system.

The Online Election System also provides separate functionalities for voters and administrators, making the election process more organized and easier to manage. Voters can register, log in, view candidate information, and cast their votes through the online interface, while administrators can manage voter details, candidate information, election details, and voting records. The system integrates frontend, backend, and database components to process user requests and maintain election-related information efficiently. Developing this project provided practical experience in web application development, database integration, authentication, validation, debugging, and testing, while applying theoretical knowledge to a practical real-world application.

1.1 OBJECTIVES OF THE INTERNSHIP

Specific Objectives

The main objective of the Online Election System is to develop a secure, user-friendly, and efficient web application for conducting and managing elections electronically.

The objectives of the project are:

- To develop a web-based application for online elections.
- To provide secure voter registration and login functionality.
- To allow registered voters to access the election system.
- To display candidate information to voters.
- To allow eligible voters to cast their votes electronically.
- To maintain voter information securely.
- To manage candidate information efficiently.
- To maintain accurate voting records.
- To reduce paperwork involved in traditional election procedures.
- To reduce manual effort in election management.
- To minimize errors in maintaining election records.
- To prevent duplicate voting through proper validation.
- To provide an easy-to-use interface for voters and administrators.
- To maintain election information in a centralized database.
- To demonstrate practical application of frontend and backend web development

- To provide a centralized platform for managing voters, candidates, and election details.
- To provide quick and easy access to election information for authorized users.
- To gain practical experience in developing and testing a complete web-based application.

1.2 MOTIVATION

The motivation behind developing the Online Election System is to make the election process easier, faster, and more organized. Traditional election procedures may depend on paper ballots, manual verification, physical queues, and manual counting, which require considerable time and administrative effort.

The proposed system provides an online platform where registered voters can log in, view candidate information, and cast their votes electronically. This reduces the dependence on paper-based election procedures and provides a convenient digital voting environment.

Another major motivation is to reduce paperwork and improve the organization of election records. Voter information, candidate information, and voting records can be stored in a centralized database, making information management easier.

The project also provides practical knowledge of web application development by using HTML, CSS, JavaScript, Node.js, Express.js, and MongoDB. It demonstrates how modern web technologies can be applied to solve real-world election management problems.

The motivation for developing this project is to provide a simple and convenient digital platform for conducting elections. By using an online system, voters can participate easily while administrators can manage election activities and records in an organized manner. The project also aims to demonstrate how modern web technologies can be used to improve traditional election processes.

2.COMPANY PROFILE

About the Company:

Fantasy Solution is an Information Technology (IT) company that focuses on providing software development and digital technology solutions to meet the growing needs of modern businesses. The company specializes in developing web

applications, software solutions, and technology-based services that help organizations improve their operational efficiency and adopt digital solutions. The primary objective of Fantasy Solution is to develop reliable, user-friendly, and efficient software applications that address real-world business requirements. The company follows modern software development practices and focuses on application design, development, database management, testing, and implementation. It also emphasizes quality, performance, maintainability, and customer satisfaction throughout the development process.

Fantasy Solution also provides industrial training and internship opportunities for students and fresh graduates. These programs help students bridge the gap between academic knowledge and practical industry requirements by providing exposure to real-time software development projects. During the internship, students can gain practical experience in areas such as frontend development, backend development, database management, REST API development, software testing, debugging, and application deployment.

As part of my internship at Fantasy Solution, I developed a Car Parking Management System using Java Full Stack technologies. The project involved designing a responsive and user-friendly interface using HTML, CSS, JavaScript,and Bootstrap, developing backend services using Java Spring Boot, integrating the MySQL database, implementing RESTful APIs, and testing the application. The system was designed to simplify parking management by providing features such as user registration and login, parking slot management, vehicle management, parking slot booking, and administrative management.

The internship provided me with valuable practical knowledge of Java Full Stack Development and helped me understand the complete Software Development Life Cycle (SDLC). It improved my programming skills, database handling, problemsolving abilities, API development knowledge, and understanding of frontendbackend integration.

Overall, the internship at Fantasy Solution provided an excellent opportunity to apply theoretical concepts in a practical software development environment. The experience helped me understand industry-oriented development practices and strengthened my technical skills in Java, Spring Boot, MySQL, HTML, CSS, JavaScript and REST API development.

Company Details:

• Company Name: Fantasy Solution

• Industry: Information Technology (IT) Services and Software Development

• Location: # 16,Samanth Plazza,Madurai Main Road,Melapudur,Trichy-1,Tamil Nadu, India

• Development Technologies: Java, Spring Boot, HTML, CSS, JavaScript, Bootstrap, MySQL, REST APIs

• Major Services: Software Development, Web Application Development, Database Solutions, Mobile Application Development, Software Testing, and Technical Training

• Internship Training: Java Full Stack Development, Web Application Development, Database Management, REST API Development, Software Testing, and Real-Time Projects

• Development Approach: User-friendly, scalable, secure, reliable, and industry-oriented software solutions

• Project Completed During Internship: Online Election System (Java Full Stack)

Services Offered:

• Software Development

• Web Application Development

• Website Design and Development

• Mobile Application Development

• Java Full Stack Development

• Database Design and Management

• REST API Development

• Digital Marketing and SEO

• Domain Registration and Web Hosting

• Technical Internship and Industrial Training

• Software Maintenance and Technical Support

Technologies Used by the Company

• Programming Languages: Java, JavaScript

• Frontend: HTML5, CSS3, JavaScript

• Backend: Java Spring Boot

• Database: MySQL

• Development Tools: Eclipse, IntelliJ IDEA, VS Code

• Build Tool: Apache Maven

• Version Control: Git and GitHub

• API Technology: RESTful Web Services

3. DAILY WORK REPORT

Day 1: Attended orientation session; briefed on internship guidelines, evaluation criteria, and reporting format. Discussed and finalized the project topic – **"Online Election System"** – with the faculty in-charge.

Day 2: Conducted requirement gathering by discussing with the mentor. Listed the stakeholders (**Voter, Administrator**) and identified the core functional requirements of the online election application.

Day 3: Carried out a literature review of existing online voting and election systems. Studied their features and documented the drawbacks of traditional manual and paper-based election procedures.

Day 4: Finalized the scope and objectives of the project. Prepared the list of hardware and software requirements needed for development.

Day 5: Set up the development environment and installed the required software and development tools for frontend and backend development. Verified the installation by running a basic web application.

Day 6: Studied web development fundamentals – HTML, CSS, and JavaScript. Understood webpage structure, styling, form handling, client-side validation, and user interaction required for the Online Election System.

Day 7: Designed the overall system architecture of the application using a client-server approach (**Presentation Layer, Backend Layer, and Database Layer**). Prepared the architecture diagram showing communication between the frontend, Node.js/Express.js backend, and MongoDB database.

Day 8: Designed the database schema by identifying entities – **Voter, Candidate, Election, Vote, and Administrator** – along with their required attributes.

Day 9: Prepared the Entity-Relationship (ER) diagram showing relationships between **Voter, Candidate, Election, and Vote** entities. Reviewed the database structure to ensure that the required election information could be stored efficiently.

Day 10: Created wireframes for the **Home, Login, Registration, Candidate, and Voting pages**. Planned the user-friendly layout and navigation structure of the Online Election System.

Day 11: Developed the **Home Page** using HTML, CSS, and JavaScript, including the navigation bar, system introduction, election information, and relevant sections for accessing the application.

Day 12: Developed the **Login Page UI** with username/email and password fields. Implemented client-side validation to ensure that the required login information was entered correctly.

Day 13: Developed the **Registration Page UI** with fields for voter details and password information. Implemented input validation to ensure that the registration data was entered correctly.

Day 14: Initialized the **Node.js and Express.js backend project**. Organized the project into suitable backend modules and configured the basic server structure required for handling requests from the Online Election System frontend.

Day 15: Created the Model (entity) classes – User, TravelPackage, and Vehicle – defining their fields and getters/setters.

Day 16: Implemented the UserController and UserService to handle new user registration, including basic input validation and duplicate-email checks.

Day 17: Implemented the AuthController for the login module, including logic in UserService to authenticate users by email and password.

Day 18: Tested the Registration and Login REST APIs using Postman. Verified success and error responses (e.g., invalid credentials, duplicate user).

Day 19: Developed the Package Module – TravelPackage model, PackageController, and PackageService – to support adding, updating, and listing travel packages.

Day 20: Developed the Vehicle Module – Vehicle model, VehicleController, and VehicleService – to manage vehicle type, seating capacity, availability, and rental cost.

Day 21: Built the Package listing and Vehicle listing pages on the frontend and connected them to the respective backend APIs using JavaScript fetch calls.

Day 22: Developed the Booking Module – Booking model, BookingController, and BookingService – to create and store booking records for packages and vehicles.

Day 23: Designed and integrated the **Voting Page** on the frontend, allowing registered voters to view available candidates, select a candidate, and submit their vote through the online interface.

Day 24: Developed the **Voting Module** to process submitted votes, validate voter information, store voting records, and update the voting status after successful vote submission.

Day 25: Designed the **Candidate Page** frontend showing candidate information and integrated it with the backend to retrieve and display the available candidates for voters.

Day 26: Developed the **Vote Validation Module**, enabling the system to verify voter eligibility and prevent a voter from submitting multiple votes for the same election.

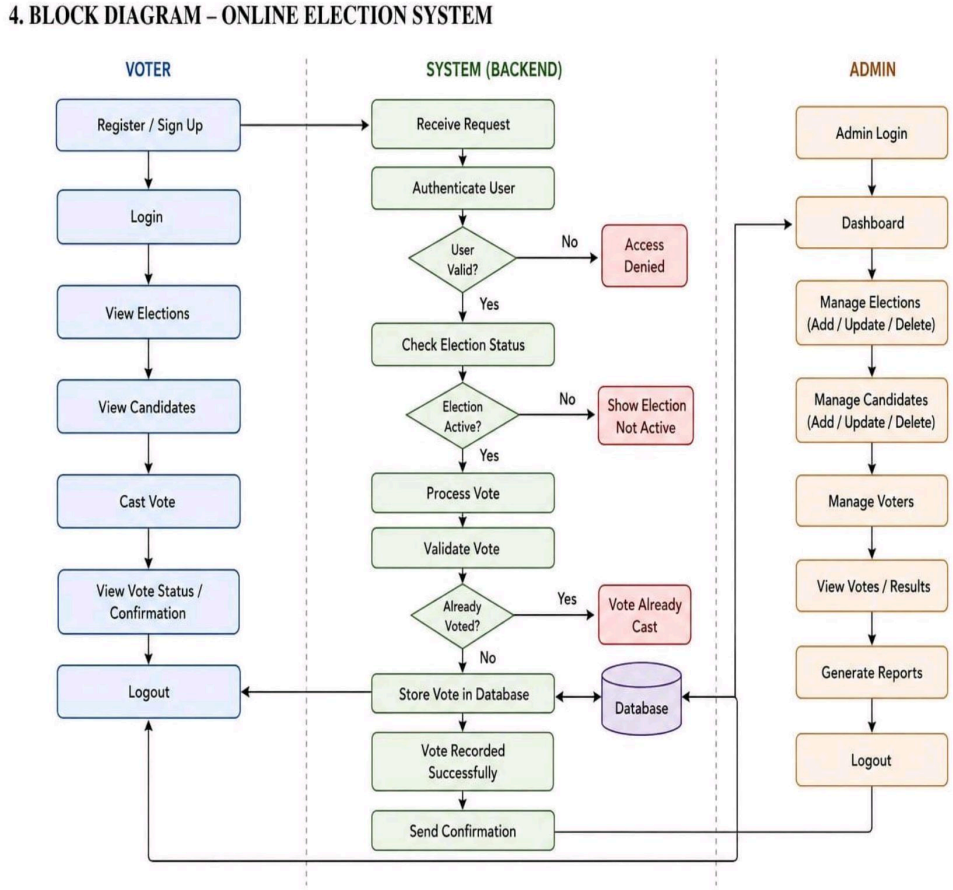
Day 27: Developed the **Voter Management Module** for managing registered voter information, checking voter status, and maintaining voter records within the system.

Day 28: Developed the **Election Management Module** to manage election information, candidate details, and election-related records through the administrator interface.

Day 29: Developed the **Admin Module** with an administrator dashboard and functionality to manage voters, candidates, elections, and voting records.

Day 30: Performed unit and integration testing of all implemented modules, fixed identified bugs, verified frontend-backend and database operations, took screenshots for documentation, and compiled the final internship report.

4.BLOCK DIAGRAM



5. ONLINE ELECTION SYSTEM

5.1 Existing System

Traditional election systems generally operate through manual and paper-based processes for voter registration, candidate management, voting, vote counting, and record maintenance.

The existing approach may have several limitations:

- **Manual voter management** – Maintaining voter information manually can consume time and effort.
- **Paper-based records** – Election information may be maintained using physical documents or separate files.

- **Time-consuming process** – Manual handling of voter and election records can increase processing time.
- **Difficulty in maintaining records** – Updating and retrieving voter information manually can be difficult.
- **Limited accessibility** – Users may need to physically participate in different stages of the election process.
- **Manual vote management** – Handling voting information manually increases administrative workload.
- **Human errors** – Manual data entry and record management can result in incorrect or duplicate information.
- **Difficulty in monitoring** – Administrators may have difficulty maintaining centralized information about voters, candidates, and votes.

These limitations create a need for a centralized web-based system that can simplify election management and provide a more organized method for handling voter and election information.

5.2 Proposed System

The proposed **Online Election System** is a web-based application that digitizes the basic election management process. It provides a centralized platform for voters and administrators.

The system provides the following features:

- **User registration** – New users can create an account by providing the required information.
- **User login** – Registered users can log in to access election functionalities.
- **Candidate display** – Users can view the available candidates.
- **Online voting** – Authenticated users can select a candidate and submit their vote.
- **Vote validation** – The system validates voting requests and checks whether a user has already voted.
- **Vote storage** – Voting information is stored in the database.
- **Candidate management** – Administrators can manage candidate information.
- **Voter management** – Administrators can view and manage voter information.
- **Centralized database** – Election-related information is maintained in MongoDB.
- **User-friendly interface** – The web interface provides simple navigation and easy access to election functions.

The proposed system reduces manual work and provides a centralized platform for managing basic election operations.

5.3 Capabilities

The Online Election System provides the following capabilities:

- **User Registration** – Allows new voters to create an account.
- **User Authentication** – Provides login functionality for registered users.
- **Candidate Management** – Allows administrators to manage candidate information.
- **Candidate Viewing** – Allows voters to view available candidates.
- **Online Voting** – Allows authenticated users to cast their vote.
- **Vote Validation** – Validates voting requests and prevents repeated voting within the implemented election flow.
- **Vote Storage** – Stores voting records in the database.
- **Voter Management** – Allows administrators to manage voter information.
- **Election Management** – Provides administrative control over election-related information.
- **Database Management** – Stores and retrieves application data using MongoDB.
- **Status Management** – Provides appropriate responses for successful and unsuccessful operations.

6. PROPOSED METHOD

6.1 User Registration

- **Allows new users to register in the system.**
- **Collects the required voter information.**
- **Validates the entered registration details.**
- **Checks for existing user information where applicable.**
- **Stores valid user information in the database.**
- **Displays an appropriate registration response.**

6.2 User Authentication

- Allows registered users to log in.
- Accepts the user's login credentials.
- Validates the entered information.
- Checks the user information against stored records.
- Provides access to the voting interface after successful authentication.
- Displays an error message when invalid credentials are entered.

6.3 Candidate Management

- Stores candidate information.

- Allows administrators to add candidate records.
- Allows candidate information to be viewed.
- Allows administrators to update candidate information where required.
- Allows removal of candidate records where applicable.

6.4 Voting Management

- Displays available candidates to authenticated voters.
- Allows a voter to select a candidate.
- Accepts the voting request.
- Validates the voter and selected candidate information.
- Stores the vote in the database.
- Provides confirmation after successful voting.

6.5 Vote Validation

- Verifies that the voter is registered.
- Checks whether the voting request contains valid information.
- Checks whether the voter has already voted.
- Rejects invalid or duplicate voting attempts.
- Allows a valid vote to proceed to storage.

6.6 Admin Management

- Provides administrative control over the system.
- Allows administrators to manage voters.
- Allows administrators to manage candidates.
- Allows administrators to view election-related information.
- Allows administrators to monitor stored voting records.
- Provides centralized management of election data.

6.7 Database Management

- Stores voter information.
- Stores candidate information.
- Stores voting information.

- Provides data retrieval and updating through the backend.
- Maintains organized election records.
- Provides centralized data management using MongoDB.

6.8 Result Management

- Retrieves stored voting information.
- Organizes vote information based on candidates.
- Allows election information to be analyzed by the administrator.
- Provides the foundation for displaying election results.

7. ALGORITHM

7.1 User Registration – Algorithm

1. User opens the registration page.
2. User enters the required registration details.
3. System validates the entered information.
4. System checks whether the user already exists.
5. If the user already exists, registration is rejected.
6. If the information is valid, a new user record is created.
7. User information is stored in the database.
8. A successful registration response is displayed.

7.2 User login – Algorithm

1. User opens the login page.
2. User enters login credentials.
3. System validates the entered fields.
4. System checks the user information in the database.
5. If the credentials are invalid, an error message is displayed.
6. If the credentials are valid, access to the system is provided.
7. The user is directed to the appropriate page.

7.3 Candidate Management – Algorithm

1. User logs into the system.
2. System verifies the user's access.

3. User opens the candidate section.
4. System requests candidate information from the backend.
5. Backend retrieves candidate records from MongoDB.
6. Candidate information is returned to the frontend.
7. Available candidates are displayed to the user.

7.4 Vote Casting – Algorithm

1. User opens the registration page.
2. User enters the required registration details.
3. System validates the entered information.
4. System checks whether the user already exists.
5. If the user already exists, registration is rejected.
6. If the information is valid, a new user record is created.
7. User information is stored in the database.
8. A successful registration response is displayed.

7.5 Vote Validation – Algorithm

1. Receive the voting request.
2. Validate the voter information.
3. Validate the selected candidate.
4. Search the database for an existing vote from the voter.
5. If an existing vote is found, reject the voting request.
6. Otherwise, accept the voting request.
7. Pass the valid vote to the storage process.

7.6 Vote Storage – Algorithm

1. Receive the validated vote.
2. Create the vote record.
3. Associate the vote with the voter and selected candidate.
4. Store the vote in MongoDB.
5. Confirm successful storage.
6. Send the response to the frontend.
7. Display the voting confirmation to the user.

7.7 Admin Operation– Algorithm

1. Administrator enters valid login credentials.
2. System verifies the administrator information.
3. Admin dashboard is displayed.
4. Administrator selects the required operation.
5. System provides options to manage voters, candidates, and election information.
6. Administrator performs the required operation.
7. System validates the changes.
8. Updated information is stored in MongoDB.
9. Updated information is displayed on the admin interface.
10. User opens the registration page.
11. User enters the required registration details.
12. System validates the entered information.
13. System checks whether the user already exists.
14. If the user already exists, registration is rejected.
15. If the information is valid, a new user record is created.
16. User information is stored in the database.
17. A successful registration response is displayed.

8.CODE IMPLEMENTATION

8.1 Login Module Code

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const app = express();
app.use(cors());
app.use(express.json());
app.use(express.static("public"));
mongoose.connect("mongodb://127.0.0.1:27017/onlineElection")
   .then(() => console.log("MongoDB Connected"))
   .catch(err => console.log(err));
const UserSchema = new mongoose.Schema({
```

```
    name: String,
    email: String,
    password: String,
    role:
type: String,
        default: "voter"
  },
    hasVoted: {
        type: Boolean,
default: false
    }
});
const CandidateSchema = new mongoose.Schema({
    name: String,
    party: String,
    symbol: String
});
const VoteSchema = new mongoose.Schema({
    voterId: mongoose.Schema.Types.ObjectId,
    candidateId: mongoose.Schema.Types.ObjectId,
    candidateName: String,
    voteDate: {
        type: Date,
        default: Date.now
    }
});
const User = mongoose.model("User", UserSchema);
const Candidate = mongoose.model("Candidate", CandidateSchema);
const Vote = mongoose.model("Vote", VoteSchema);
// USER REGISTRATION
app.post("/api/register", async (req, res) => {
    try {
        const { name, email, password } = req.body;
```

```
      const existingUser = await User.findOne({ email });
      if (existingUser) {
          return res.status(400).json({
              message: "Email already registered"
          });
      }
      const user = new User({
          name,
          email,
          password,
          role: "voter"
      });
     await user.save();
     res.status(201).json({
          message: "Registration successful"
      });
     } catch (error) {
      res.status(500).json({
          message: "Registration failed"
      });
   }
});
// USER LOGIN
app.post("/api/login", async (req, res) => {
   try {
      const { email, password } = req.body;

      const user = await User.findOne({
          email,
          password
      });
      if (!user) {
```

```
            return res.status(401).json({
                message: "Invalid email or password"
            });
        }
        res.json({
            message: "Login successful",
            user: {
                id: user._id,
                name: user.name,
                email: user.email,
                role: user.role,
                hasVoted: user.hasVoted
            }
        });
       } catch (error) {
        res.status(500).json({
            message: "Login failed"
        });
    }
});
// GET CANDIDATES
app.get("/api/candidates", async (req, res) => {
    try {
        const candidates = await Candidate.find();
        res.json(candidates);
    } catch (error) {
        res.status(500).json({
            message: "Unable to retrieve candidates"
        });
    }
});
// ADD CANDIDATE
app.post("/api/candidates", async (req, res) => {
```

```
    try {
        const { name, party, symbol } = req.body;
         const candidate = new Candidate({
            name,
            party,
            symbol
        });
        await candidate.save();
        res.status(201).json({
            message: "Candidate added successfully",
            candidate
        }
        } catch (error) {
        res.status(500).json({
            message: "Failed to add candidate"
        });
    }
});
// CAST VOTE
app.post("/api/vote", async (req, res) => {
    try {
        const { voterId, candidateId } = req.body;
         const voter = await User.findById(voterId);

        if (!voter) {
            return res.status(404).json({
                message: "Voter not found"
            });
        }
         if (voter.hasVoted) {
            return res.status(400).json({
                message: "You have already voted"
            });
```

```
        }
         const candidate = await Candidate.findById(candidateId);
          if (!candidate) {
            return res.status(404).json({
                message: "Candidate not found"
            });
            const vote = new Vote({
            voterId: voter._id,
            candidateId: candidate._id,
            candidateName: candidate.name
        });
       await vote.save();
      voter.hasVoted = true;
        await voter.save()
      res.json({
            message: "Vote cast successfully"
        });

    } catch (error) {
        res.status(500).json({
            message: "Voting failed"
        });
    }
});
// GET ALL VOTES
app.get("/api/votes", async (req, res) => {
    try {
        const votes = await Vote.find();
        res.json(votes);
    } catch (error) {
        res.status(500).json({
            message: "Unable to retrieve votes"
        });
```

```
    }
});
// GET ALL USERS
app.get("/api/users", async (req, res) => {
    try {
        const users = await User.find();
        res.json(users);
    } catch (error) {
        res.status(500).json({
            message: "Unable to retrieve users"
        });
    }
});



const PORT = 5000;
app.listen(PORT, () => {
    console.log(`Server running on http://localhost:${PORT}`);
});
```

8.2 Registration Module Code

```
<!DOCTYPE html>
<html>
<head>
    <title>Online Election - Registration</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
    <h2>Voter Registration</h2>
 <input type="text" id="name" placeholder="Enter Name">
    <input type="email" id="email" placeholder="Enter Email">
    <input type="password" id="password" placeholder="Enter Password">
<button onclick="register()">Register</button>
```

```
 <p id="message"></p>
</div>
<script>
async function register() {

    const name = document.getElementById("name").value;
    const email = document.getElementById("email").value;
    const password = document.getElementById("password").value;
    const response = await fetch("/api/register", {
        method: "POST",
        headers: {
            "Content-Type": "application/json"
        },
        body: JSON.stringify({
            name,
            email,
            password
        })
    });
  const data = await response.json();
 document.getElementById("message").innerText = data.message;
}
</script>
</body>
</html>
```

8.3 Login Module Code

```
<!DOCTYPE html>
<html>
<head>
    <title>Online Election - Login</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
```

```
<div class="container">
<h2>Voter Login</h2>
 <input type="email" id="email" placeholder="Email">
 <input type="password" id="password" placeholder="Password">
<button onclick="login()">Login</button>
<p id="message"></p>
</div>
<script>
async function login() {
    const email = document.getElementById("email").value;
    const password = document.getElementById("password").value;
    const response = await fetch("/api/login", {
        method: "POST",
        headers: {
            "Content-Type": "application/json"
        },
        body: JSON.stringify({
            email,
            password
        })
    });
 const data = await response.json()
 if (response.ok)
localStorage.setItem(
            "user",
            JSON.stringify(data.user)
        )
 window.location.href = "candidates.html";
  } else {
          document.getElementById("message").innerText =
           data.message;
    }
}
```

```
</script>
</body>
</html>
```

8.4 Candidate And Voting Module Code

```
<!DOCTYPE html>
<html>
<head>
    <title>Online Election - Candidates</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
<h2>Available Candidates</h2>
   <div id="candidateList"></div>
    <p id="message"></p>
</div>
<script>
const user = JSON.parse(localStorage.getItem("user"));
if (!user) {
    window.location.href = "login.html";
}
   async function loadCandidates() {
    const response = await fetch("/api/candidates");
    const candidates = await response.json();
    const list = document.getElementById("candidateList");

    candidates.forEach(candidate => {
    const div = document.createElement("div");
   div.className = "candidate";
  div.innerHTML = `
            <h3>${candidate.name}</h3>
            <p>Party: ${candidate.party}</p>
            <p>Symbol: ${candidate.symbol}</p>
```

```
<button onclick="vote('${candidate._id}')">
                Vote
            </button>
list.appendChild(div);
    });
}
async function vote(candidateId) {
 if (user.hasVoted) {
 document.getElementById("message").innerText =
            "You have already voted.";
  return;
    }
 const response = await fetch("/api/vote", {
 method: "POST",
 headers: {
            "Content-Type": "application/json"
        },
body: JSON.stringify({
            voterId: user.id,
            candidateId: candidateId
        })
    });
  const data = await response.json();
 document.getElementById("message").innerText =
        data.message;
 if (response.ok) {
    user.hasVoted = true;
    localStorage.setItem(
            "user",
            JSON.stringify(user)
        );
    }
}
```

```
loadCandidates();
</script>
</body>
</html>
```

8.5 Admin Module Code

```
<!DOCTYPE html>
<html>
<head>
    <title>Online Election - Admin</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
 <h2>Admin Dashboard</h2>
 <h3>Add Candidate</h3>
<input type="text" id="name"
          placeholder="Candidate Name"
 <input type="text" id="party"
          placeholder="Political Party">

    <input type="text" id="symbol"
          placeholder="Candidate Symbol">
 <button onclick="addCandidate()">
        Add Candidate
    </button>
  <p id="message"></p>
 <hr>
  <h3>Candidate List</h3>
<div id="candidateList"></div>
</div>
<script>
async function addCandidate() {
  const name =
```

```
        document.getElementById("name").value;
  const party =
        document.getElementById("party").value;
   const symbol =
        document.getElementById("symbol").value;

    const response = await fetch(
        "/api/candidates",
        {
            method: "POST",
            headers: {
                "Content-Type": "application/json"
            },
         body: JSON.stringify({
                name,
                party,
                symbol
            })
        }
    );
   const data = await response.json();
    document.getElementById("message").innerText =
        data.message;
        loadCandidates();
}



async function loadCandidates() {
const response =
        await fetch("/api/candidates");

    const candidates =
        await response.json();
```

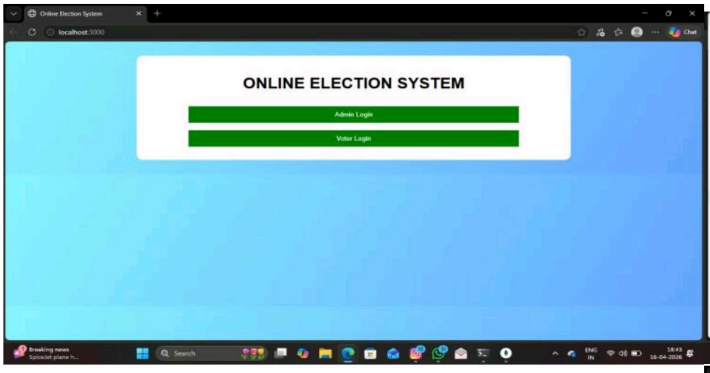
```
        const list =
        document.getElementById("candidateList");
        list.innerHTML = "";
    candidates.forEach(candidate => {
    list.innerHTML += `
            <div class="candidate">
                <h4>${candidate.name}</h4>
                <p>${candidate.party}</p>
                <p>${candidate.symbol}</p>
            </div>
        `;
    });
}
loadCandidates();
</script>
</body>
</html>
```

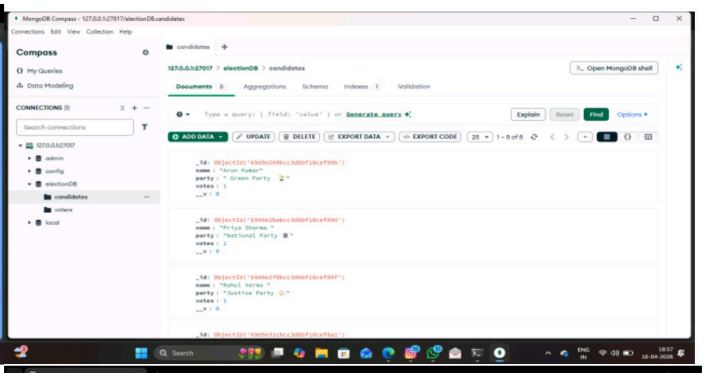
8.6 Database Management Module Code

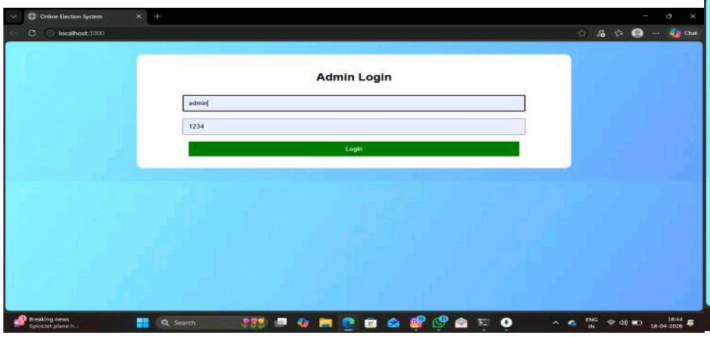
```
body {
    font-family: Arial, sans-serif;
    background: #f2f2f2;
    margin: 0;
    padding: 30px;
}
.container {
    width: 400px;
    margin: auto;
    background: white;
    padding: 25px;
    border-radius: 10px;
    box-shadow: 0 0 10px rgba(0,0,0,0.1);
}
h2 {
```

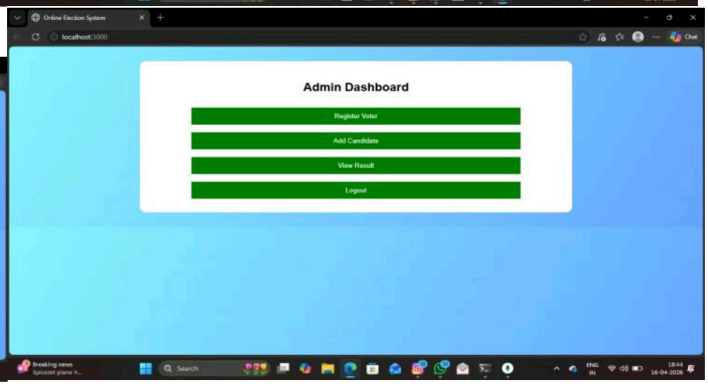
```
    text-align: center;
}
input {
    width: 100%;
    padding: 10px;
    margin: 8px 0;
    box-sizing: border-box;
}
button {
    width: 100%;
    padding: 10px;
    margin-top: 10px;
    background: #333;
    color: white;
    border: none;
    cursor: pointer;
}
button:hover {
    opacity: 0.8;
}
.candidate {
    border: 1px solid #ddd;
    padding: 15px;
    margin-top: 15px;
    border-radius: 5px;
}
#message {
    text-align: center;
    margin-top: 15px;
```

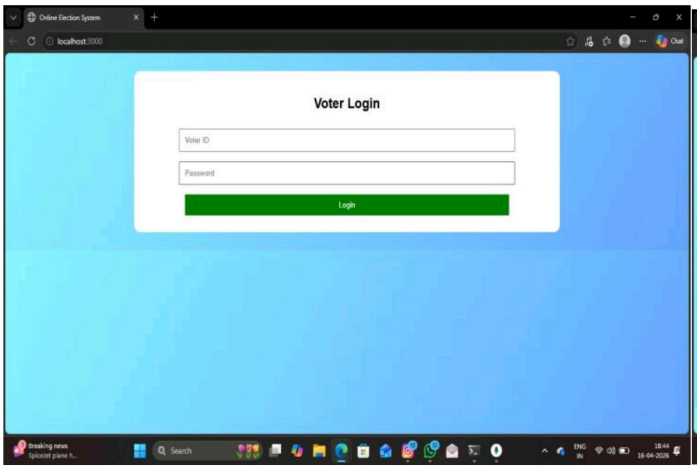
9.SAMPLE OUTPUT

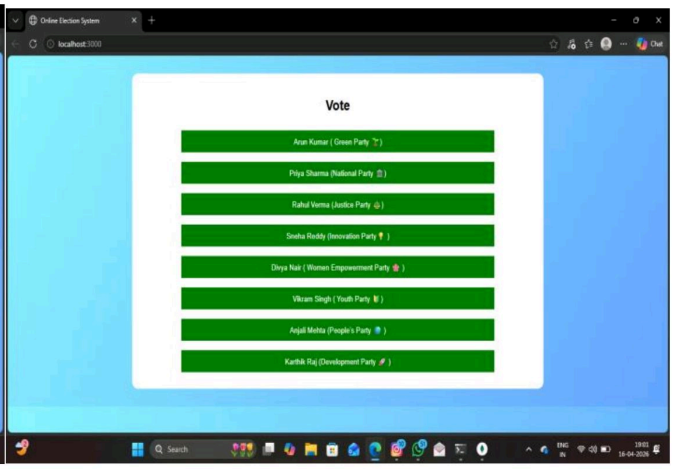


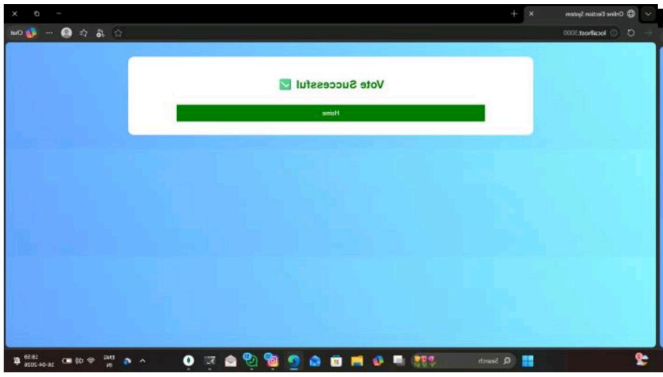


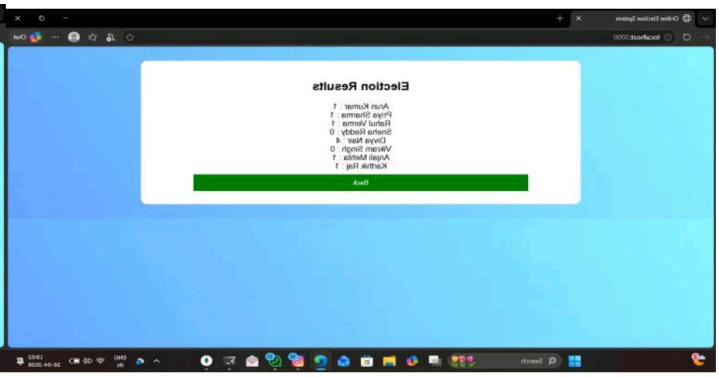












10.ANALYSIS

Testing of the Online Election System was carried out module by module to verify the functional correctness and overall behavior of the application.Functional testing was performed for user registration, login, candidate viewing, vote casting, vote validation, database operations, and administrative functions. Different input conditions such as empty fields, invalid login credentials, duplicate registration details, invalid voting requests, and repeated voting attempts were considered during testing.The frontend and backend communication was tested to ensure that requests were properly sent from the web interface to the Node.js and Express.js backend. Database operations were also tested to verify that voter, candidate, and voting information was stored and retrieved correctly.Usability analysis showed that the web interface provides a simple navigation structure for users. The voting process can be completed through a series of straightforward steps, reducing the complexity associated with manual election procedures.Overall, the analysis indicates that the Online Election System provides an organized platform for managing basic election activities and demonstrates the integration of frontend development, backend development, API communication, and database management.

11.CONCLUSION

The **Online Election System** successfully demonstrates how web technologies can be used to digitize and simplify basic election management activities. The system provides a centralized platform for user registration, authentication, candidate viewing, voting, vote validation, and election data management.The frontend developed using HTML, CSS, and JavaScript provides a simple and user-friendly interface, while Node.js and Express.js handle the backend operations and API communication. MongoDB provides centralized storage for voter, candidate, and voting information.The project helped provide practical knowledge of web application development, JavaScript programming, Node.js, Express.js, MongoDB, REST APIs, database operations, frontend-backend integration, validation, and software testing.The developed system provides a foundation for a more advanced online election platform. Additional security, authentication, auditing, and scalability features can be incorporated in future versions to make the system more suitable for larger election environments.

12.FUTURE WORK

- Implementation of stronger user authentication and authorization.
- Integration of OTP-based voter verification.
- Implementation of JWT-based authentication.
- Addition of role-based access control for voters and administrators.
- Implementation of stronger vote security and audit mechanisms.
- Development of an improved administrator dashboard.
- Addition of graphical election result visualization.
- Implementation of real-time election result monitoring.

- Addition of email or SMS notifications.
- Development of a mobile application for Android and iOS.
- Cloud deployment for improved scalability and availability.
- Implementation of improved database security and backup mechanisms.
- Addition of election scheduling and automatic election closing.
- Implementation of detailed election reports and analytics.
- Improvement of accessibility and responsive design.

13. REFERENCES

1. W3Schools – HTML, CSS, and JavaScript learning resources.
2. MDN Web Docs – Web development, HTML, CSS, and JavaScript documentation.
3. Node.js Documentation – Node.js runtime and application development.
4. Express.js Documentation – Web framework and REST API development.
5. MongoDB Documentation – Database management and MongoDB concepts.
6. JavaScript Documentation – Programming concepts and web application development.
7. GeeksforGeeks – Node.js, Express.js, MongoDB, JavaScript, and web development concepts.

FANTASY SOLUTION

Fins Us Passion...

SOFTWARE DEVELOPMENT AND TRAINING

Certificate of Accomplishment

This is certify that Mr./Miss. JEYA SANORA FRESHIA J (810023205703)

Student of UNIVERSITY COLLEGE OF ENGINEERING TIRUCHIRAPALLI (BIT CAMPUS)

underwent Inplant Training / Internship / Course on FULL STACK DEVELOPMENT

concern from 01/06/2026 to 30/06/2026 issued on 30/06/2026

Technical Incharge

Manager

CEO

#16, Samnath Plazza, 3rd Floor, Madurai Main Road, Melapudur, Trichy-1.
0431 356 3660 fantazysolution@gmail.com www.fantasysolution.in

ATTENDANCE CERTIFICATE FOR INDUSTRIAL TRAINING / INTERNSHIP

(Attendance Certificate to be signed by the competent authority of the industry mentioning the period of Industrial Training / Internship)

The Student Mr./ Ms. **JEYA SANORA FRESHIA J** (Reg.No.) **810023205703** ,

Studying **FULL STACK DEVELOPMENT** programme at **UNIVERSITY COLLEGE OF ENGINEERING (BIT CAMPUS), TIRUCHIRAPALLI** College in semester **7th** at Department of **INFORMATION TECHNOLOGY** has attended Industrial Training / Internship from **01/06/2026** to **30/06/2026** . It is certified that he / she has completed / not completed the Industrial Training / Internship in our **FANTASY SOLUTION** organization / institution.



Signature

Name and Designation of the Officer
Seal of the Organization

Fantasy Solution

Find ur Passion...

Date: 01.06.2026

J. JEYA SANORA FRESHIA,
Reg No : 810023205703,
Department of Information Technology,
University College of Engineering,
Bharathidasan Institute of Technology (BIT) Campus,
Tiruchirappalli - 620 024.

Dear J. Jeya sanora freshia,

Sub: Internship

On behalf of **FANTASY SOLUTION.** We would like to inform you that you have been accepted as one of our interns. We are pleased to inform you that you have been accepted as per the company requirements for the internship with **FULL STACK AND WEB DEVELOPMENT.**

Your internship will begin from **01st June 2026 to 30th June 2026.** You would assign to various tasks which relates to the project assigned to you after whom your performance would be assessed and appraised.

FANTASY SOLUTION TRICHY.1

Authorized Signature
(With Seal)

#16, Samnath Plazza, 3rd Floor, Melapudur, Trichy - 620 001.
Ph : 0431-4250535 | Mob : 90430 95535
www.fantasysolution.in | info@fantasysolution.in